SIMATS ENGINEERING

Department of Electrical and Electronics Engineering

ASSESSMENT TOOL – 3

Course Code:	ECA1418	Course Title:	Embedded System
CO Assessed:	CO : 2	Assessment:	AT-3
Weightage:	40%	Total Marks:	100
CO1 Statement:	Apply Embedded C programming concepts, including structured design, real-time constraint handling, and SystemC-based modeling, to develop and analyze embedded applications for industrial use cases.		
Student Name:	M.Nandhini	Reg. No.:	192472318
Section / Batch:	slot-c	Date:	15-08-26

Question Type	Description	Questions	Total Marks
	Coding Challenge	Q – Q	100

Code of Conduct

I m.nandhini(192472318) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: m.nandhini

RUBRICS FOR MARKS CALCULATION

Task	(%)	(%)	(%)	(%)	Total Marks (Each -)
				Overall Total ( )	

Faculty In-charge	Course Coordinator	Head of Department
Name: Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

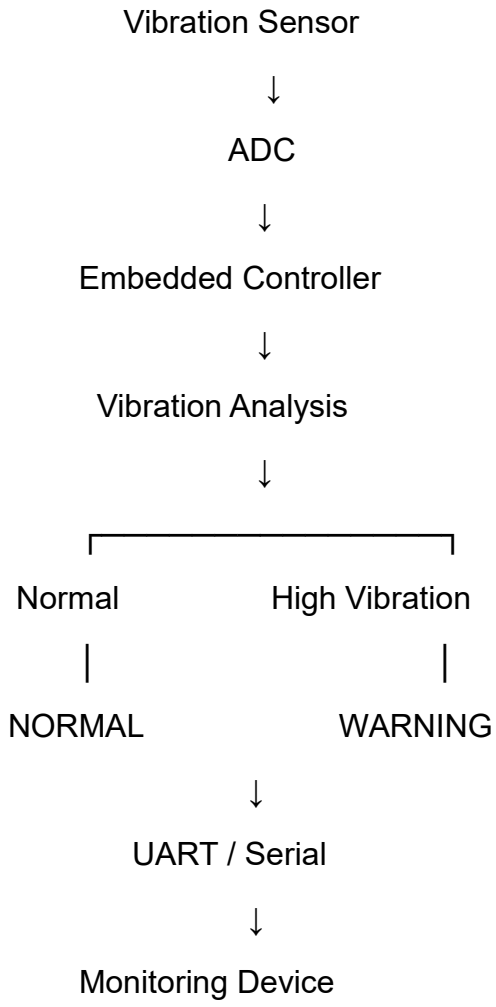
QUESTION-1

Design of a Vibration Monitoring Node with Data Acquisition and Serial Transmission Using Embedded C

1. Aim

To design an Embedded C program for a vibration monitoring node that acquires vibration data from a sensor, checks the vibration level against a predefined threshold, and transmits the measured data and status through a serial communication interface.

2. System Block Diagram

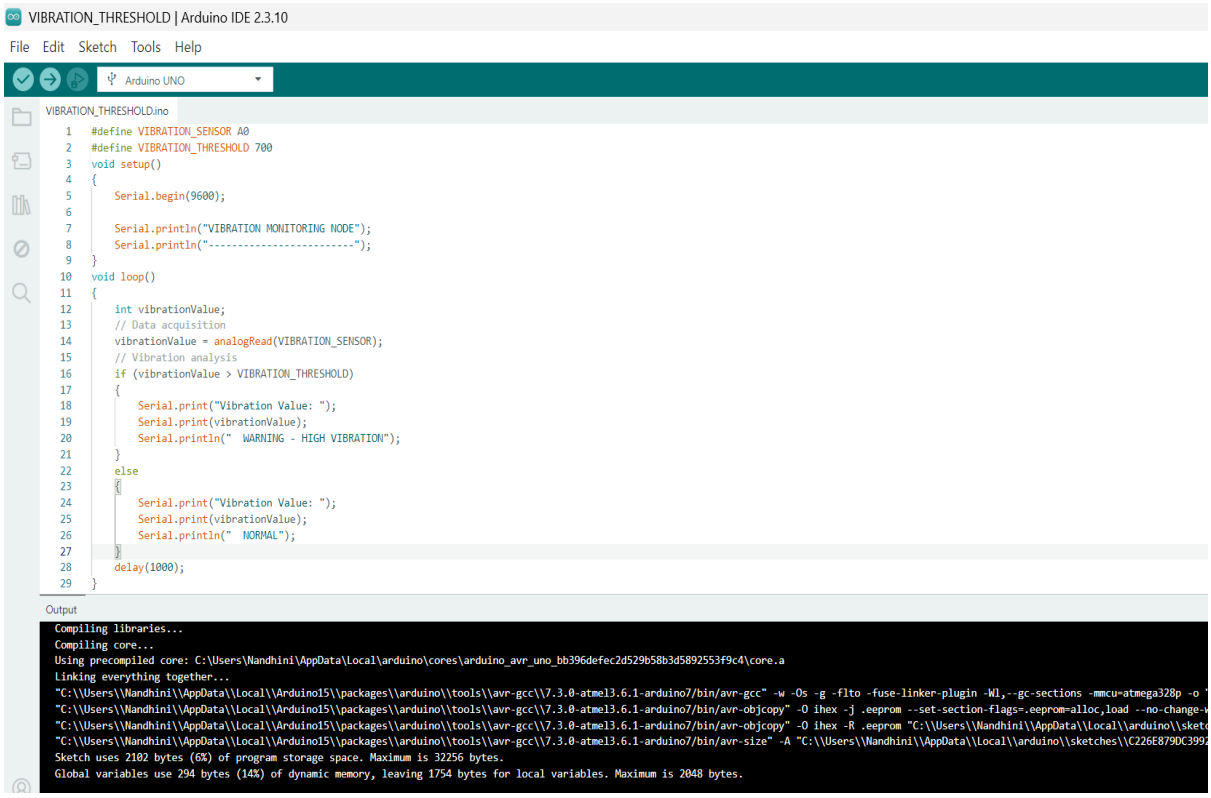


3. Working Principle

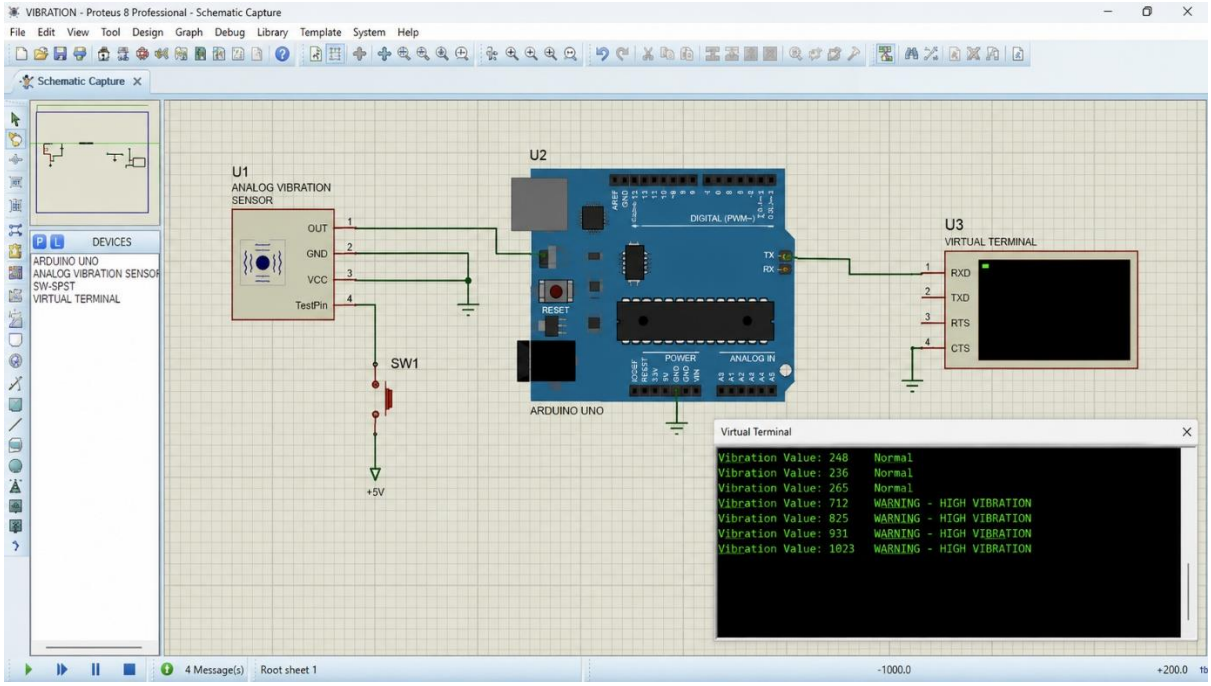
1. The vibration sensor detects mechanical vibration from the machine.
2. The sensor output is converted into a digital value using an ADC.
3. The embedded controller reads the ADC value.
4. The measured value is compared with a predefined vibration threshold.
5. If the vibration is within the safe limit, the system reports NORMAL.

6. If the vibration exceeds the threshold, the system reports WARNING.
7. The vibration value and status are transmitted through the serial interface.

4. Embedded C Program



5. Sample Output



6. Design Justification

- uint16_t is used for vibration/ADC values because sensor readings are non-negative integer values.
- A threshold value of 700 is defined using a macro so that it can easily be changed according to the machine specification.
- The program is divided into separate functions for data acquisition, processing, and serial transmission, making the code structured and easier to maintain.
- In an actual embedded system, read_vibration_sensor() would interface with the ADC hardware, while serial_transmit() would use UART/USART.
- Serial transmission allows the vibration data to be sent to a PC, display, or industrial monitoring system.
- If the vibration exceeds the threshold, a warning is generated so that abnormal machine conditions can be detected early.

7. Real-Time Consideration

The monitoring node should periodically acquire vibration data rather than waiting indefinitely. In an actual implementation, a timer or hardware interrupt can trigger data acquisition at fixed intervals. A timeout mechanism can also be used to detect sensor or communication failure.

8. Result

Thus, a vibration monitoring node was designed using Embedded C to acquire vibration data, analyze the measured value against a threshold, and transmit the vibration value and machine status through serial communication.

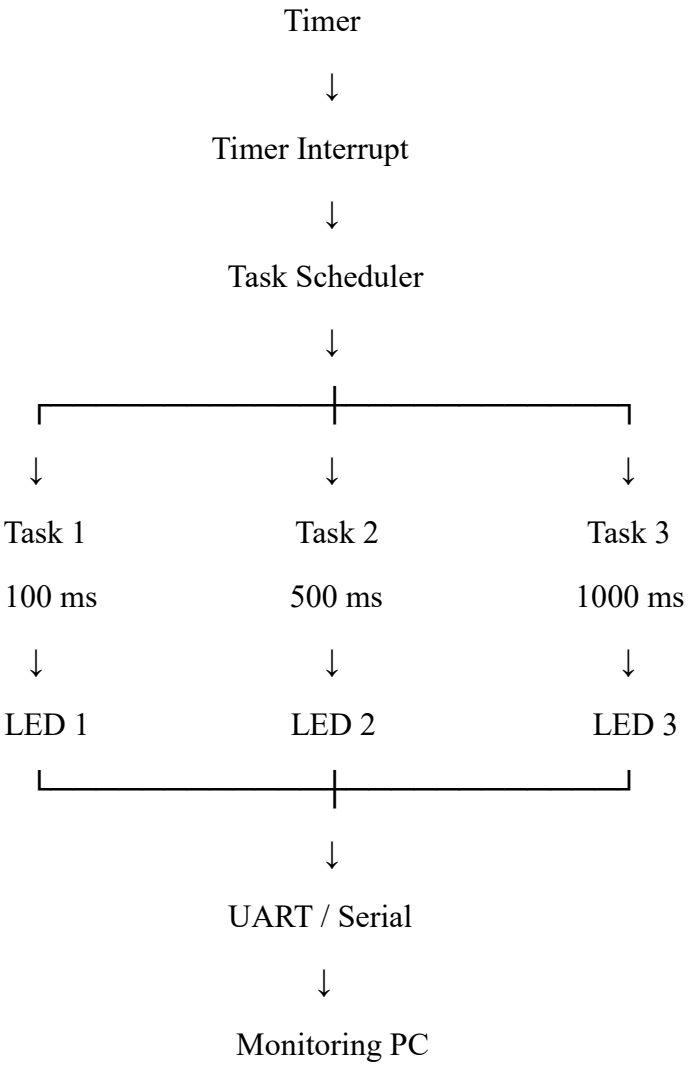
QUESTION-2

Design of a Real-Time Embedded C Application Scheduling Multiple Tasks Using Timer Interrupts

1. Aim

To design a real-time Embedded C application that schedules and executes multiple tasks at different time intervals using timer interrupts, demonstrating periodic task execution and efficient real-time scheduling.

2. System Block Diagram

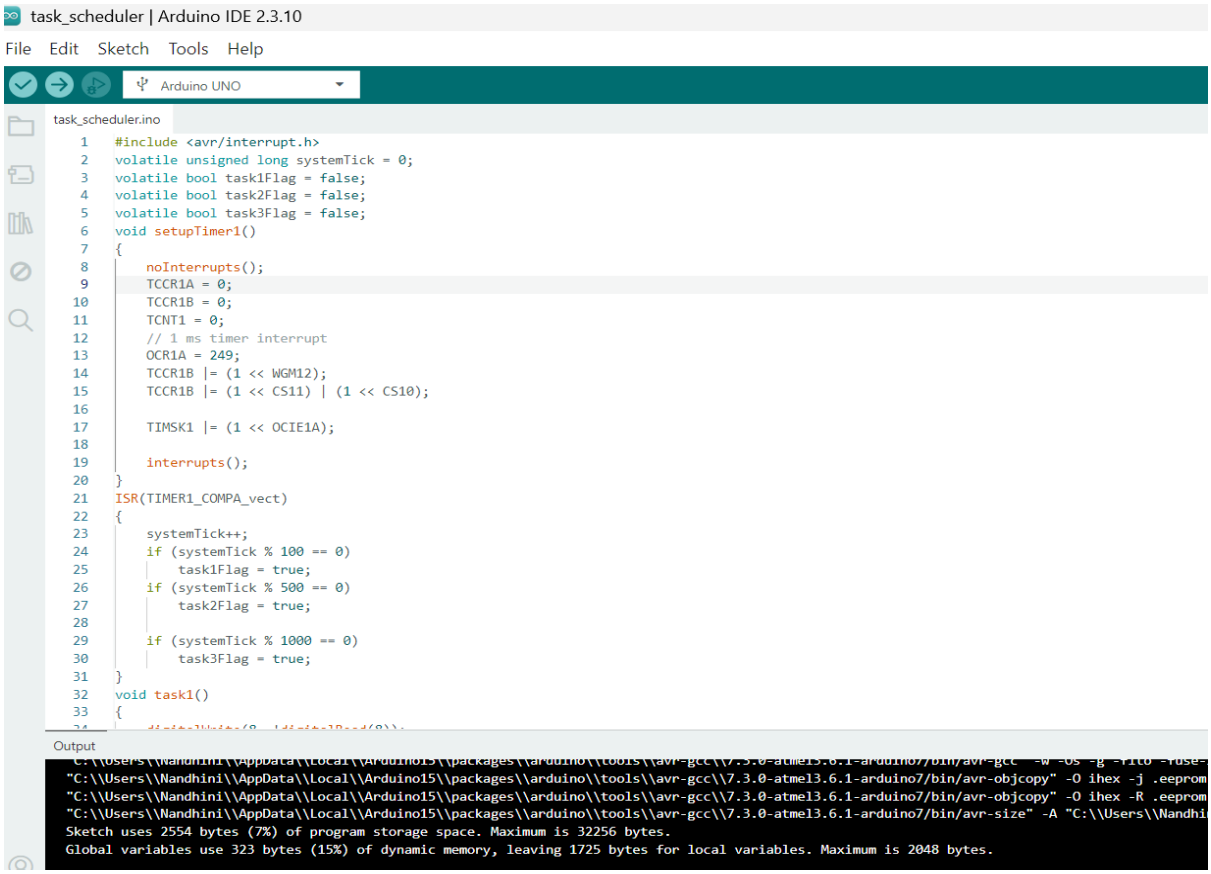


3. Working Principle

1. A hardware Timer1 is configured to generate an interrupt every 1 ms.
2. Each timer interrupt increments the system tick counter.
3. Task 1 is scheduled every 100 ms.
4. Task 2 is scheduled every 500 ms.
5. Task 3 is scheduled every 1000 ms.

6. The timer interrupt sets a flag whenever a task reaches its scheduled time.
7. The main program checks the task flags and executes the corresponding task.
8. LEDs indicate the execution of each task.
9. The task execution status is transmitted through serial communication.

4. Embedded C Program



task_scheduler | Arduino IDE 2.3.10

File Edit Sketch Tools Help

Arduino UNO

task_scheduler.ino

```
}

void task3()
{
    digitalWrite(10, !digitalRead(10));

    Serial.println("Task 3: Control Operation");
}
void setup()
{
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);

    Serial.begin(9600);

    Serial.println("REAL-TIME TASK SCHEDULER");
    Serial.println("------------------------");

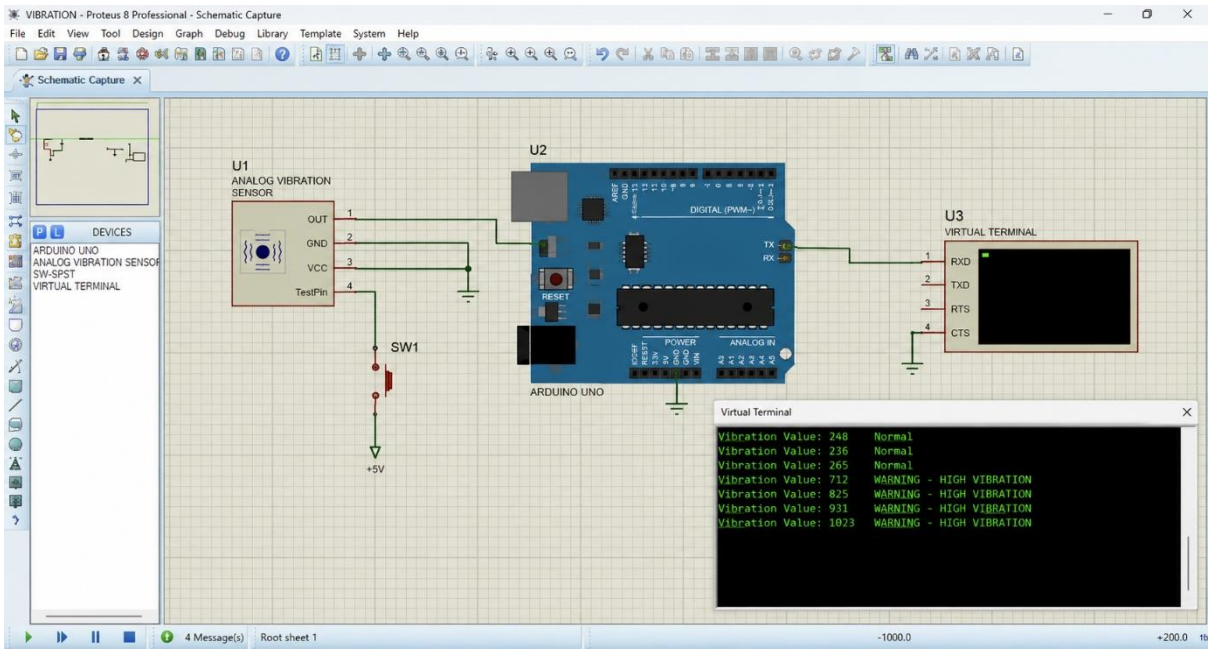
    setupTimer1();
}

void loop()
{
    if (task1Flag)
    {
        task1Flag = false;
        task1();
    }
    if (task2Flag)
    {
        task2Flag = false;
        task2();
```

Output

```
"C:\\Users\\Nandhini\\AppData\\Local\\Arduino15\\packages\\arduino\\tools\\avr-gcc\\7.3.0-atmel3.6.1-arduino7/bin/avr-objcopy" -O ihex -j .eeprom --se
"C:\\Users\\Nandhini\\AppData\\Local\\Arduino15\\packages\\arduino\\tools\\avr-gcc\\7.3.0-atmel3.6.1-arduino7/bin/avr-objcopy" -O ihex -R .eeprom "C:\
"C:\\Users\\Nandhini\\AppData\\Local\\Arduino15\\packages\\arduino\\tools\\avr-gcc\\7.3.0-atmel3.6.1-arduino7/bin/avr-size" -A "C:\\Users\\Nandhini\\A
Sketch uses 2554 bytes (7%) of program storage space. Maximum is 32256 bytes.
Global variables use 323 bytes (15%) of dynamic memory, leaving 1725 bytes for local variables. Maximum is 2048 bytes.
```

5. Sample Output



6. Design Justification

- Timer1 is used to provide accurate periodic timing for task scheduling.

- A 1 ms timer interrupt provides the basic time reference for the application.
- Task 1, Task 2 and Task 3 are scheduled at 100 ms, 500 ms and 1000 ms respectively.
- Volatile flags are used because their values are modified inside the interrupt service routine.
- The interrupt service routine only sets task flags, while the actual tasks are executed in the main loop. This avoids performing lengthy operations inside the interrupt.
- Separate functions are used for each task, providing structured and maintainable code.
- LEDs are used to visually indicate task execution, while serial communication displays the task status.
- Timer-based scheduling provides more reliable periodic execution than using blocking delays.

7. Real-Time Consideration

The application uses a hardware timer interrupt to provide regular task scheduling. Since each task has a predefined execution period, the system can perform multiple operations at different time intervals. The use of timer interrupts prevents the processor from continuously waiting for a delay and allows other tasks to be executed when required.

8. Result

Thus, a real-time Embedded C application was designed and implemented to schedule multiple periodic tasks using timer interrupts. The three tasks were executed at different time intervals, with LED indication and serial transmission used to demonstrate the task execution.